

Summative Assessment 2

DSC1107 Data Mining and Wrangling

Sindayen, Lorenzo Danilo V.

2026-07-20

GitHub Link:
<https://github.com/ChewyGnome/DSC1107>

Part I — Machine Learning Models for Customer Churn

Introduction

In this assessment, supervised machine learning techniques will be applied to predict customer churn using the Orange Telecom Churn Dataset. The objective of this study is to develop and compare regression-based and tree-based classification models that are capable of identifying customers who are likely to discontinue their telecommunications service. The model performance will then be evaluated using Accuracy, Precision, Recall, F1-score, and ROC-AUC to determine the most effective approach for customer churn prediction.

1.0 Dataset Familiarization & Preparation

Import Training and Testing Datasets

```
train_raw <- read_csv("churn-bigml-80.csv") %>%  
  clean_names()  
  
test_raw <- read_csv("churn-bigml-20.csv") %>%  
  clean_names()
```

Display Dataset Dimensions

```
dim(train_raw)
```

```
## [1] 2666  20
```

```
dim(test_raw)
```

```
## [1] 667 20
```

Display Dataset Structure

```
str(train_raw)
```

```
## spc_tbl_ [2,666 x 20] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
## $ state           : chr [1:2666] "KS" "OH" "NJ" "OH" ...
## $ account_length  : num [1:2666] 128 107 137 84 75 118 121 147 141 74 ...
## $ area_code       : num [1:2666] 415 415 415 408 415 510 510 415 415 415 ...
## $ international_plan : chr [1:2666] "No" "No" "No" "Yes" ...
## $ voice_mail_plan  : chr [1:2666] "Yes" "Yes" "No" "No" ...
## $ number_vmail_messages : num [1:2666] 25 26 0 0 0 0 24 0 37 0 ...
## $ total_day_minutes : num [1:2666] 265 162 243 299 167 ...
## $ total_day_calls   : num [1:2666] 110 123 114 71 113 98 88 79 84 127 ...
## $ total_day_charge  : num [1:2666] 45.1 27.5 41.4 50.9 28.3 ...
## $ total_eve_minutes : num [1:2666] 197.4 195.5 121.2 61.9 148.3 ...
## $ total_eve_calls   : num [1:2666] 99 103 110 88 122 101 108 94 111 148 ...
## $ total_eve_charge  : num [1:2666] 16.78 16.62 10.3 5.26 12.61 ...
## $ total_night_minutes : num [1:2666] 245 254 163 197 187 ...
## $ total_night_calls  : num [1:2666] 91 103 104 89 121 118 118 96 97 94 ...
## $ total_night_charge : num [1:2666] 11.01 11.45 7.32 8.86 8.41 ...
## $ total_intl_minutes : num [1:2666] 10 13.7 12.2 6.6 10.1 6.3 7.5 7.1 11.2 9.1 ...
## $ total_intl_calls   : num [1:2666] 3 3 5 7 3 6 7 6 5 5 ...
## $ total_intl_charge  : num [1:2666] 2.7 3.7 3.29 1.78 2.73 1.7 2.03 1.92 3.02 2.46 ...
## $ customer_service_calls: num [1:2666] 1 1 0 2 3 0 3 0 0 0 ...
## $ churn             : logi [1:2666] FALSE FALSE FALSE FALSE FALSE FALSE ...
## - attr(*, "spec")=
## .. cols(
## ..   State = col_character(),
## ..   'Account length' = col_double(),
## ..   'Area code' = col_double(),
## ..   'International plan' = col_character(),
## ..   'Voice mail plan' = col_character(),
## ..   'Number vmail messages' = col_double(),
## ..   'Total day minutes' = col_double(),
## ..   'Total day calls' = col_double(),
## ..   'Total day charge' = col_double(),
## ..   'Total eve minutes' = col_double(),
## ..   'Total eve calls' = col_double(),
## ..   'Total eve charge' = col_double(),
## ..   'Total night minutes' = col_double(),
## ..   'Total night calls' = col_double(),
## ..   'Total night charge' = col_double(),
## ..   'Total intl minutes' = col_double(),
## ..   'Total intl calls' = col_double(),
## ..   'Total intl charge' = col_double(),
## ..   'Customer service calls' = col_double(),
## ..   Churn = col_logical()
```

```
## .. )
## - attr(*, "problems")=<externalptr>
```

```
glimpse(train_raw)
```

```
## Rows: 2,666
## Columns: 20
## $ state <chr> "KS", "OH", "NJ", "OH", "OK", "AL", "MA", "MO", ~
## $ account_length <dbl> 128, 107, 137, 84, 75, 118, 121, 147, 141, 74, ~
## $ area_code <dbl> 415, 415, 415, 408, 415, 510, 510, 415, 415, 41~
## $ international_plan <chr> "No", "No", "No", "Yes", "Yes", "Yes", "No", "Y~
## $ voice_mail_plan <chr> "Yes", "Yes", "No", "No", "No", "No", "Yes", "N~
## $ number_vmail_messages <dbl> 25, 26, 0, 0, 0, 0, 24, 0, 37, 0, 0, 0, 0, 27, ~
## $ total_day_minutes <dbl> 265.1, 161.6, 243.4, 299.4, 166.7, 223.4, 218.2~
## $ total_day_calls <dbl> 110, 123, 114, 71, 113, 98, 88, 79, 84, 127, 96~
## $ total_day_charge <dbl> 45.07, 27.47, 41.38, 50.90, 28.34, 37.98, 37.09~
## $ total_eve_minutes <dbl> 197.4, 195.5, 121.2, 61.9, 148.3, 220.6, 348.5,~
## $ total_eve_calls <dbl> 99, 103, 110, 88, 122, 101, 108, 94, 111, 148, ~
## $ total_eve_charge <dbl> 16.78, 16.62, 10.30, 5.26, 12.61, 18.75, 29.62,~
## $ total_night_minutes <dbl> 244.7, 254.4, 162.6, 196.9, 186.9, 203.9, 212.6~
## $ total_night_calls <dbl> 91, 103, 104, 89, 121, 118, 118, 96, 97, 94, 12~
## $ total_night_charge <dbl> 11.01, 11.45, 7.32, 8.86, 8.41, 9.18, 9.57, 9.5~
## $ total_intl_minutes <dbl> 10.0, 13.7, 12.2, 6.6, 10.1, 6.3, 7.5, 7.1, 11.~
## $ total_intl_calls <dbl> 3, 3, 5, 7, 3, 6, 7, 6, 5, 5, 2, 5, 6, 4, 3, 5,~
## $ total_intl_charge <dbl> 2.70, 3.70, 3.29, 1.78, 2.73, 1.70, 2.03, 1.92,~
## $ customer_service_calls <dbl> 1, 1, 0, 2, 3, 0, 3, 0, 0, 0, 1, 3, 4, 1, 3, 1,~
## $ churn <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE~
```

```
str(test_raw)
```

```
## spc_tbl_ [667 x 20] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
## $ state : chr [1:667] "LA" "IN" "NY" "SC" ...
## $ account_length : num [1:667] 117 65 161 111 49 36 65 119 10 68 ...
## $ area_code : num [1:667] 408 415 415 415 510 408 415 415 408 415 ...
## $ international_plan : chr [1:667] "No" "No" "No" "No" ...
## $ voice_mail_plan : chr [1:667] "No" "No" "No" "No" ...
## $ number_vmail_messages : num [1:667] 0 0 0 0 0 30 0 0 0 0 ...
## $ total_day_minutes : num [1:667] 184 129 333 110 119 ...
## $ total_day_calls : num [1:667] 97 137 67 103 117 128 120 114 112 70 ...
## $ total_day_charge : num [1:667] 31.4 21.9 56.6 18.8 20.3 ...
## $ total_eve_minutes : num [1:667] 352 228 318 137 215 ...
## $ total_eve_calls : num [1:667] 80 83 97 102 109 80 122 117 66 164 ...
## $ total_eve_charge : num [1:667] 29.9 19.4 27 11.7 18.3 ...
## $ total_night_minutes : num [1:667] 216 209 161 190 179 ...
## $ total_night_calls : num [1:667] 90 111 128 105 90 109 118 91 57 103 ...
## $ total_night_charge : num [1:667] 9.71 9.4 7.23 8.53 8.04 ...
## $ total_intl_minutes : num [1:667] 8.7 12.7 5.4 7.7 11.1 14.5 13.2 8.8 11.4 12.1 ...
## $ total_intl_calls : num [1:667] 4 6 9 6 1 6 5 3 6 3 ...
## $ total_intl_charge : num [1:667] 2.35 3.43 1.46 2.08 3 3.92 3.56 2.38 3.08 3.27 ...
## $ customer_service_calls: num [1:667] 1 4 4 2 1 0 3 5 2 3 ...
## $ churn : logi [1:667] FALSE TRUE TRUE FALSE FALSE FALSE ...
## - attr(*, "spec")=
## .. cols(
```

```
## .. State = col_character(),
## .. 'Account length' = col_double(),
## .. 'Area code' = col_double(),
## .. 'International plan' = col_character(),
## .. 'Voice mail plan' = col_character(),
## .. 'Number vmail messages' = col_double(),
## .. 'Total day minutes' = col_double(),
## .. 'Total day calls' = col_double(),
## .. 'Total day charge' = col_double(),
## .. 'Total eve minutes' = col_double(),
## .. 'Total eve calls' = col_double(),
## .. 'Total eve charge' = col_double(),
## .. 'Total night minutes' = col_double(),
## .. 'Total night calls' = col_double(),
## .. 'Total night charge' = col_double(),
## .. 'Total intl minutes' = col_double(),
## .. 'Total intl calls' = col_double(),
## .. 'Total intl charge' = col_double(),
## .. 'Customer service calls' = col_double(),
## .. Churn = col_logical()
## .. )
## - attr(*, "problems")=<externalptr>
```

```
glimpse(test_raw)
```

```
## Rows: 667
## Columns: 20
## $ state <chr> "LA", "IN", "NY", "SC", "HI", "AK", "MI", "ID", ~
## $ account_length <dbl> 117, 65, 161, 111, 49, 36, 65, 119, 10, 68, 74, ~
## $ area_code <dbl> 408, 415, 415, 415, 510, 408, 415, 415, 408, 41~
## $ international_plan <chr> "No", "No", "No", "No", "No", "No", "No", "No", ~
## $ voice_mail_plan <chr> "No", "No", "No", "No", "No", "Yes", "No", "No"~
## $ number_vmail_messages <dbl> 0, 0, 0, 0, 0, 30, 0, 0, 0, 0, 33, 0, 0, 29, 0, ~
## $ total_day_minutes <dbl> 184.5, 129.1, 332.9, 110.4, 119.3, 146.3, 211.3~
## $ total_day_calls <dbl> 97, 137, 67, 103, 117, 128, 120, 114, 112, 70, ~
## $ total_day_charge <dbl> 31.37, 21.95, 56.59, 18.77, 20.28, 24.87, 35.92~
## $ total_eve_minutes <dbl> 351.6, 228.5, 317.8, 137.3, 215.1, 162.5, 162.6~
## $ total_eve_calls <dbl> 80, 83, 97, 102, 109, 80, 122, 117, 66, 164, 96~
## $ total_eve_charge <dbl> 29.89, 19.42, 27.01, 11.67, 18.28, 13.81, 13.82~
## $ total_night_minutes <dbl> 215.8, 208.8, 160.6, 189.6, 178.7, 129.3, 134.7~
## $ total_night_calls <dbl> 90, 111, 128, 105, 90, 109, 118, 91, 57, 103, 9~
## $ total_night_charge <dbl> 9.71, 9.40, 7.23, 8.53, 8.04, 5.82, 6.06, 6.44, ~
## $ total_intl_minutes <dbl> 8.7, 12.7, 5.4, 7.7, 11.1, 14.5, 13.2, 8.8, 11.~
## $ total_intl_calls <dbl> 4, 6, 9, 6, 1, 6, 5, 3, 6, 3, 3, 3, 3, 7, 4, 4, ~
## $ total_intl_charge <dbl> 2.35, 3.43, 1.46, 2.08, 3.00, 3.92, 3.56, 2.38, ~
## $ customer_service_calls <dbl> 1, 4, 4, 2, 1, 0, 3, 5, 2, 3, 2, 0, 2, 0, 0, 0, ~
## $ churn <lgl> FALSE, TRUE, TRUE, FALSE, FALSE, FALSE, FALSE, ~
```

Summary Statistics

```
summary(train_raw)
```

```
##      state      account_length  area_code  international_plan
## Length:2666   Min.   : 1.0    Min.   :408.0   Length:2666
## Class :character 1st Qu.: 73.0   1st Qu.:408.0   Class :character
## Mode  :character Median :100.0   Median :415.0   Mode  :character
##                Mean   :100.6   Mean   :437.4
##                3rd Qu.:127.0   3rd Qu.:510.0
##                Max.   :243.0   Max.   :510.0
## voice_mail_plan  number_vmail_messages total_day_minutes total_day_calls
## Length:2666     Min.   : 0.000      Min.   : 0.0      Min.   : 0.0
## Class :character 1st Qu.: 0.000      1st Qu.:143.4    1st Qu.: 87.0
## Mode  :character Median : 0.000      Median :179.9    Median :101.0
##                Mean   : 8.022      Mean   :179.5    Mean   :100.3
##                3rd Qu.:19.000      3rd Qu.:215.9    3rd Qu.:114.0
##                Max.   :50.000      Max.   :350.8    Max.   :160.0
## total_day_charge total_eve_minutes total_eve_calls total_eve_charge
## Min.   : 0.00    Min.   : 0.0      Min.   : 0      Min.   : 0.00
## 1st Qu.:24.38    1st Qu.:165.3    1st Qu.: 87     1st Qu.:14.05
## Median :30.59    Median :200.9    Median :100     Median :17.08
## Mean   :30.51    Mean   :200.4    Mean   :100     Mean   :17.03
## 3rd Qu.:36.70    3rd Qu.:235.1    3rd Qu.:114     3rd Qu.:19.98
## Max.   :59.64    Max.   :363.7    Max.   :170     Max.   :30.91
## total_night_minutes total_night_calls total_night_charge total_intl_minutes
## Min.   : 43.7     Min.   : 33.0     Min.   : 1.970    Min.   : 0.00
## 1st Qu.:166.9     1st Qu.: 87.0     1st Qu.: 7.513    1st Qu.: 8.50
## Median :201.2     Median :100.0     Median : 9.050    Median :10.20
## Mean   :201.2     Mean   :100.1     Mean   : 9.053    Mean   :10.24
## 3rd Qu.:236.5     3rd Qu.:113.0     3rd Qu.:10.640    3rd Qu.:12.10
## Max.   :395.0     Max.   :166.0     Max.   :17.770    Max.   :20.00
## total_intl_calls total_intl_charge customer_service_calls churn
## Min.   : 0.000    Min.   :0.000     Min.   :0.000     Mode :logical
## 1st Qu.: 3.000    1st Qu.:2.300     1st Qu.:1.000     FALSE:2278
## Median : 4.000    Median :2.750     Median :1.000     TRUE :388
## Mean   : 4.467    Mean   :2.764     Mean   :1.563
## 3rd Qu.: 6.000    3rd Qu.:3.270     3rd Qu.:2.000
## Max.   :20.000    Max.   :5.400     Max.   :9.000
```

```
summary(test_raw)
```

```
##      state      account_length  area_code  international_plan
## Length:667     Min.   : 1.0    Min.   :408.0   Length:667
## Class :character 1st Qu.: 76.0   1st Qu.:408.0   Class :character
## Mode  :character Median :102.0   Median :415.0   Mode  :character
##                Mean   :102.8   Mean   :436.2
##                3rd Qu.:128.0   3rd Qu.:415.0
##                Max.   :232.0   Max.   :510.0
## voice_mail_plan  number_vmail_messages total_day_minutes total_day_calls
## Length:667     Min.   : 0.000      Min.   : 25.9     Min.   : 30.0
## Class :character 1st Qu.: 0.000      1st Qu.:146.2     1st Qu.: 87.5
## Mode  :character Median : 0.000      Median :178.3     Median :101.0
##                Mean   : 8.408      Mean   :180.9     Mean   :100.9
```

```
##           3rd Qu.:20.000           3rd Qu.:220.7           3rd Qu.:115.0
##           Max.      :51.000           Max.      :334.3           Max.      :165.0
## total_day_charge total_eve_minutes total_eve_calls total_eve_charge
## Min.      : 4.40      Min.      : 48.1      Min.      : 37.0      Min.      : 4.09
## 1st Qu.:24.86      1st Qu.:171.1      1st Qu.: 88.0      1st Qu.:14.54
## Median :30.31      Median :203.7      Median :101.0      Median :17.31
## Mean      :30.76      Mean      :203.4      Mean      :100.5      Mean      :17.29
## 3rd Qu.:37.52      3rd Qu.:236.4      3rd Qu.:113.0      3rd Qu.:20.09
## Max.      :56.83      Max.      :361.8      Max.      :168.0      Max.      :30.75
## total_night_minutes total_night_calls total_night_charge total_intl_minutes
## Min.      : 23.2      Min.      : 42.0      Min.      : 1.040      Min.      : 0.00
## 1st Qu.:167.9      1st Qu.: 86.0      1st Qu.: 7.560      1st Qu.: 8.60
## Median :201.6      Median :100.0      Median : 9.070      Median :10.50
## Mean      :199.7      Mean      :100.1      Mean      : 8.986      Mean      :10.24
## 3rd Qu.:231.5      3rd Qu.:113.5      3rd Qu.:10.420      3rd Qu.:12.05
## Max.      :367.7      Max.      :175.0      Max.      :16.550      Max.      :18.30
## total_intl_calls total_intl_charge customer_service_calls churn
## Min.      : 0.000      Min.      :0.000      Min.      :0.000      Mode :logical
## 1st Qu.: 3.000      1st Qu.:2.320      1st Qu.:1.000      FALSE:572
## Median : 4.000      Median :2.840      Median :1.000      TRUE :95
## Mean      : 4.528      Mean      :2.765      Mean      :1.564
## 3rd Qu.: 6.000      3rd Qu.:3.255      3rd Qu.:2.000
## Max.      :18.000      Max.      :4.940      Max.      :8.000
```

Check Missing Values

```
missing_train <- tibble(
  Variable = names(train_raw),
  Missing_Values = colSums(is.na(train_raw))
)

missing_test <- tibble(
  Variable = names(test_raw),
  Missing_Values = colSums(is.na(test_raw))
)

kable(
  missing_train,
  caption = "Training Dataset Missing Values"
)
```

Table 1: Training Dataset Missing Values

Variable	Missing_Values
state	0
account_length	0
area_code	0
international_plan	0
voice_mail_plan	0
number_vmail_messages	0
total_day_minutes	0

Variable	Missing_Values
total_day_calls	0
total_day_charge	0
total_eve_minutes	0
total_eve_calls	0
total_eve_charge	0
total_night_minutes	0
total_night_calls	0
total_night_charge	0
total_intl_minutes	0
total_intl_calls	0
total_intl_charge	0
customer_service_calls	0
churn	0

```
kable(
  missing_test,
  caption = "Testing Dataset Missing Values"
)
```

Table 2: Testing Dataset Missing Values

Variable	Missing_Values
state	0
account_length	0
area_code	0
international_plan	0
voice_mail_plan	0
number_vmail_messages	0
total_day_minutes	0
total_day_calls	0
total_day_charge	0
total_eve_minutes	0
total_eve_calls	0
total_eve_charge	0
total_night_minutes	0
total_night_calls	0
total_night_charge	0
total_intl_minutes	0
total_intl_calls	0
total_intl_charge	0
customer_service_calls	0
churn	0

Notes: The missing value analysis shows that both the training and testing datasets contain no missing values across all variables. Since the datasets are complete, no additional data imputation or removal of incomplete observations is required before proceeding with data preprocessing and model development.

Remove Non-Predictive Variables

```
train_clean <- train_raw %>%  
  select(-state)  
  
test_clean <- test_raw %>%  
  select(-state)
```

Notes: The state variable was removed because it served as an identifier rather than a meaningful predictor of customer churn. Excluding this variable reduces unnecessary model complexity and allows the machine learning models to focus on customer service usage and behavioral characteristics.

Convert Churn into Binary Values

```
train_clean <- train_clean %>%  
  mutate(  
    churn = ifelse(churn == TRUE, 1, 0)  
  )  
  
test_clean <- test_clean %>%  
  mutate(  
    churn = ifelse(churn == TRUE, 1, 0)  
  )  
  
table(train_clean$churn)
```

```
##  
##      0      1  
## 2278   388
```

```
table(test_clean$churn)
```

```
##  
##      0      1  
## 572    95
```

Notes: The target variable churn was converted into binary values, where 0 represents customers who remained with the company and 1 represents customers who churned. The class distribution shows that non-churning customers outnumber churning customers in both datasets, suggesting that a moderate class imbalance that should be considered during model evaluation.

One-Hot Encode Categorical Variables

```
dummy_model <- dummyVars(  
  churn ~ .,  
  data = train_clean  
)
```



```

x_train <- predict(
  dummy_model,
  newdata = train_clean
) %>%
  as.data.frame()

x_test <- predict(
  dummy_model,
  newdata = test_clean
) %>%
  as.data.frame()

train_processed <- cbind(
  churn = train_clean$churn,
  x_train
)

test_processed <- cbind(
  churn = test_clean$churn,
  x_test
)

```

Display Processed Dataset

```
glimpse(train_processed)
```

```

## Rows: 2,666
## Columns: 21
## $ churn                <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ account_length      <dbl> 128, 107, 137, 84, 75, 118, 121, 147, 141, 74, ~
## $ area_code            <dbl> 415, 415, 415, 408, 415, 510, 510, 415, 415, 41~
## $ international_planNo <dbl> 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, ~
## $ international_planYes <dbl> 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, ~
## $ voice_mail_planNo    <dbl> 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, ~
## $ voice_mail_planYes   <dbl> 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, ~
## $ number_vmail_messages <dbl> 25, 26, 0, 0, 0, 0, 24, 0, 37, 0, 0, 0, 0, 27, ~
## $ total_day_minutes    <dbl> 265.1, 161.6, 243.4, 299.4, 166.7, 223.4, 218.2~
## $ total_day_calls      <dbl> 110, 123, 114, 71, 113, 98, 88, 79, 84, 127, 96~
## $ total_day_charge     <dbl> 45.07, 27.47, 41.38, 50.90, 28.34, 37.98, 37.09~
## $ total_eve_minutes    <dbl> 197.4, 195.5, 121.2, 61.9, 148.3, 220.6, 348.5, ~
## $ total_eve_calls      <dbl> 99, 103, 110, 88, 122, 101, 108, 94, 111, 148, ~
## $ total_eve_charge     <dbl> 16.78, 16.62, 10.30, 5.26, 12.61, 18.75, 29.62, ~
## $ total_night_minutes  <dbl> 244.7, 254.4, 162.6, 196.9, 186.9, 203.9, 212.6~
## $ total_night_calls    <dbl> 91, 103, 104, 89, 121, 118, 118, 96, 97, 94, 12~
## $ total_night_charge   <dbl> 11.01, 11.45, 7.32, 8.86, 8.41, 9.18, 9.57, 9.5~
## $ total_intl_minutes   <dbl> 10.0, 13.7, 12.2, 6.6, 10.1, 6.3, 7.5, 7.1, 11.~
## $ total_intl_calls     <dbl> 3, 3, 5, 7, 3, 6, 7, 6, 5, 5, 2, 5, 6, 4, 3, 5, ~
## $ total_intl_charge    <dbl> 2.70, 3.70, 3.29, 1.78, 2.73, 1.70, 2.03, 1.92, ~
## $ customer_service_calls <dbl> 1, 1, 0, 2, 3, 0, 3, 0, 0, 0, 1, 3, 4, 1, 3, 1, ~

```

```
glimpse(test_processed)
```

```
## Rows: 667
## Columns: 21
## $ churn <dbl> 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, ~
## $ account_length <dbl> 117, 65, 161, 111, 49, 36, 65, 119, 10, 68, 74, ~
## $ area_code <dbl> 408, 415, 415, 415, 510, 408, 415, 415, 408, 41~
## $ international_planNo <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
## $ international_planYes <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ voice_mail_planNo <dbl> 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, ~
## $ voice_mail_planYes <dbl> 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, ~
## $ number_vmail_messages <dbl> 0, 0, 0, 0, 0, 30, 0, 0, 0, 0, 33, 0, 0, 29, 0, ~
## $ total_day_minutes <dbl> 184.5, 129.1, 332.9, 110.4, 119.3, 146.3, 211.3~
## $ total_day_calls <dbl> 97, 137, 67, 103, 117, 128, 120, 114, 112, 70, ~
## $ total_day_charge <dbl> 31.37, 21.95, 56.59, 18.77, 20.28, 24.87, 35.92~
## $ total_eve_minutes <dbl> 351.6, 228.5, 317.8, 137.3, 215.1, 162.5, 162.6~
## $ total_eve_calls <dbl> 80, 83, 97, 102, 109, 80, 122, 117, 66, 164, 96~
## $ total_eve_charge <dbl> 29.89, 19.42, 27.01, 11.67, 18.28, 13.81, 13.82~
## $ total_night_minutes <dbl> 215.8, 208.8, 160.6, 189.6, 178.7, 129.3, 134.7~
## $ total_night_calls <dbl> 90, 111, 128, 105, 90, 109, 118, 91, 57, 103, 9~
## $ total_night_charge <dbl> 9.71, 9.40, 7.23, 8.53, 8.04, 5.82, 6.06, 6.44, ~
## $ total_intl_minutes <dbl> 8.7, 12.7, 5.4, 7.7, 11.1, 14.5, 13.2, 8.8, 11.~
## $ total_intl_calls <dbl> 4, 6, 9, 6, 1, 6, 5, 3, 6, 3, 3, 3, 3, 7, 4, 4, ~
## $ total_intl_charge <dbl> 2.35, 3.43, 1.46, 2.08, 3.00, 3.92, 3.56, 2.38, ~
## $ customer_service_calls <dbl> 1, 4, 4, 2, 1, 0, 3, 5, 2, 3, 2, 0, 2, 0, 0, ~
```

2.0 Exploratory Data Analysis

Churn Rate Analysis

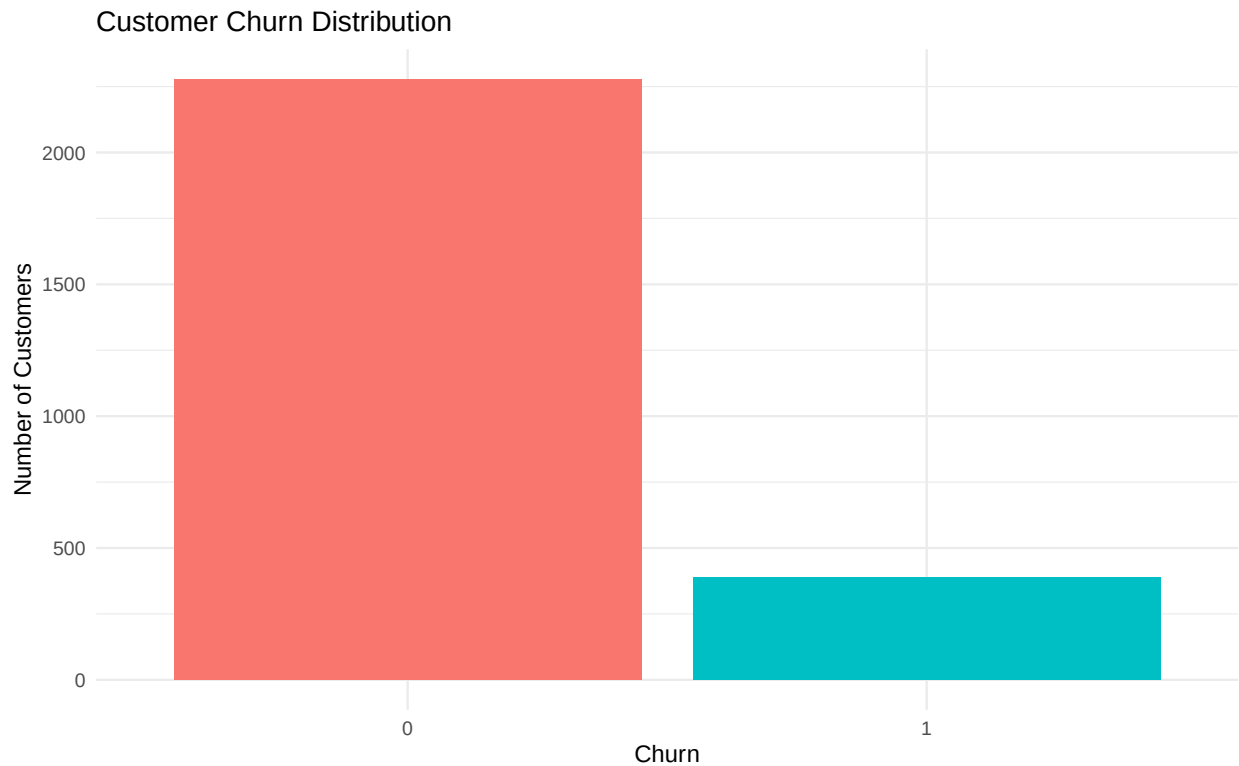
```
train_clean %>%
  count(churn) %>%
  mutate(
    Percentage = round(n / sum(n) * 100, 2)
  ) %>%
  kable(
    caption = "Customer Churn Distribution"
  )
```

Table 3: Customer Churn Distribution

churn	n	Percentage
0	2278	85.45
1	388	14.55

```
ggplot(train_clean,
  aes(x = factor(churn),
    fill = factor(churn))) +
```

```
geom_bar() +
labs(
  title = "Customer Churn Distribution",
  x = "Churn",
  y = "Number of Customers"
) +
theme_minimal() +
theme(
  legend.position = "none"
)
```

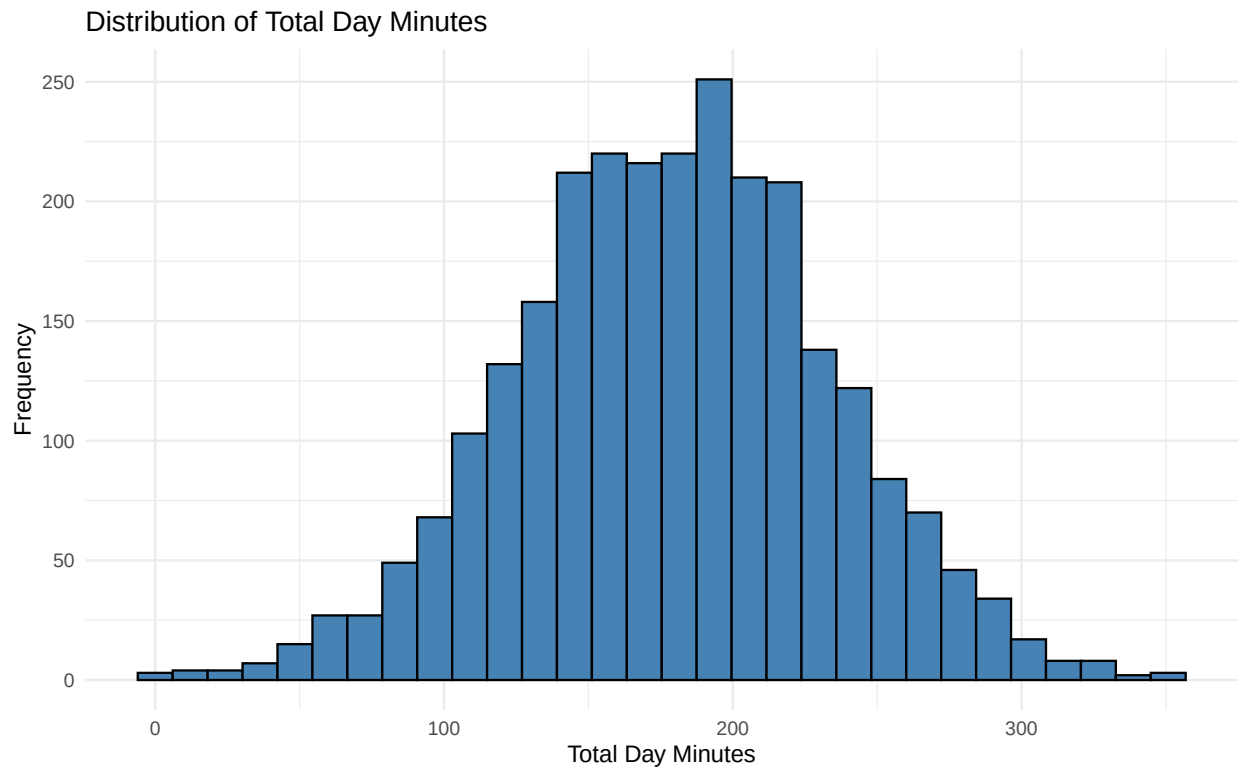


Notes: From the results it is shown that 85.45% of customers remained with the company, while only 14.55% churned. This meant that customer churn is relatively uncommon in the dataset, which results in a relatively skewed classification problem.

Distribution of Important Variables

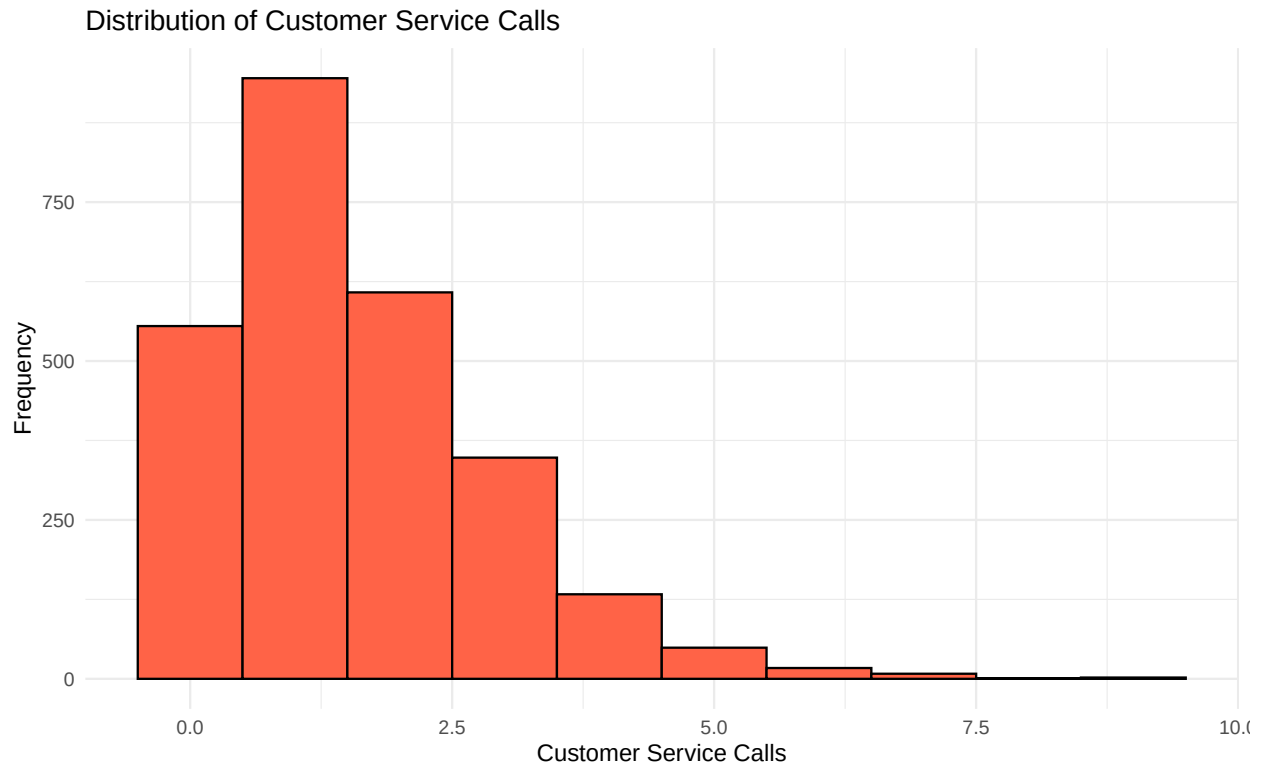
```
ggplot(train_clean,
  aes(x = total_day_minutes)) +
geom_histogram(
  bins = 30,
  fill = "steelblue",
  color = "black"
) +
labs(
  title = "Distribution of Total Day Minutes",
```

```
x = "Total Day Minutes",
y = "Frequency"
) +
theme_minimal()
```



Notes: The distribution of total day minutes is shown to be bell-shaped, with most customers using between 150 and 220 minutes during the day. Aside from a few customers also exhibited unusually low or high usage, suggesting the presence of some extreme observations from both ends of the low and high minutes. In addition to this

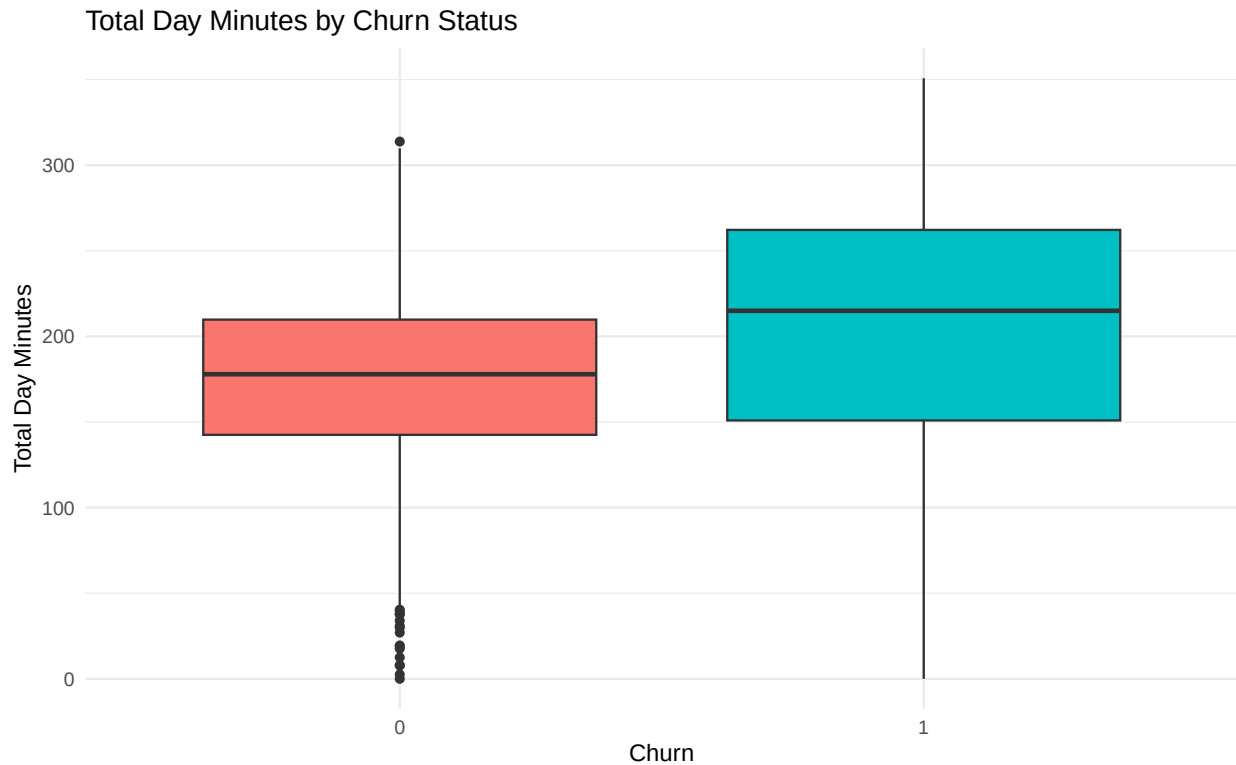
```
ggplot(train_clean,
  aes(x = customer_service_calls)) +
  geom_histogram(
    bins = 10,
    fill = "tomato",
    color = "black"
  ) +
  labs(
    title = "Distribution of Customer Service Calls",
    x = "Customer Service Calls",
    y = "Frequency"
  ) +
  theme_minimal()
```



Notes: Most customers contacted customer service between 0 and 3 times, while only a small number made frequent service calls. The distribution is positively skewed, indicating that repeated customer service interactions are relatively uncommon.

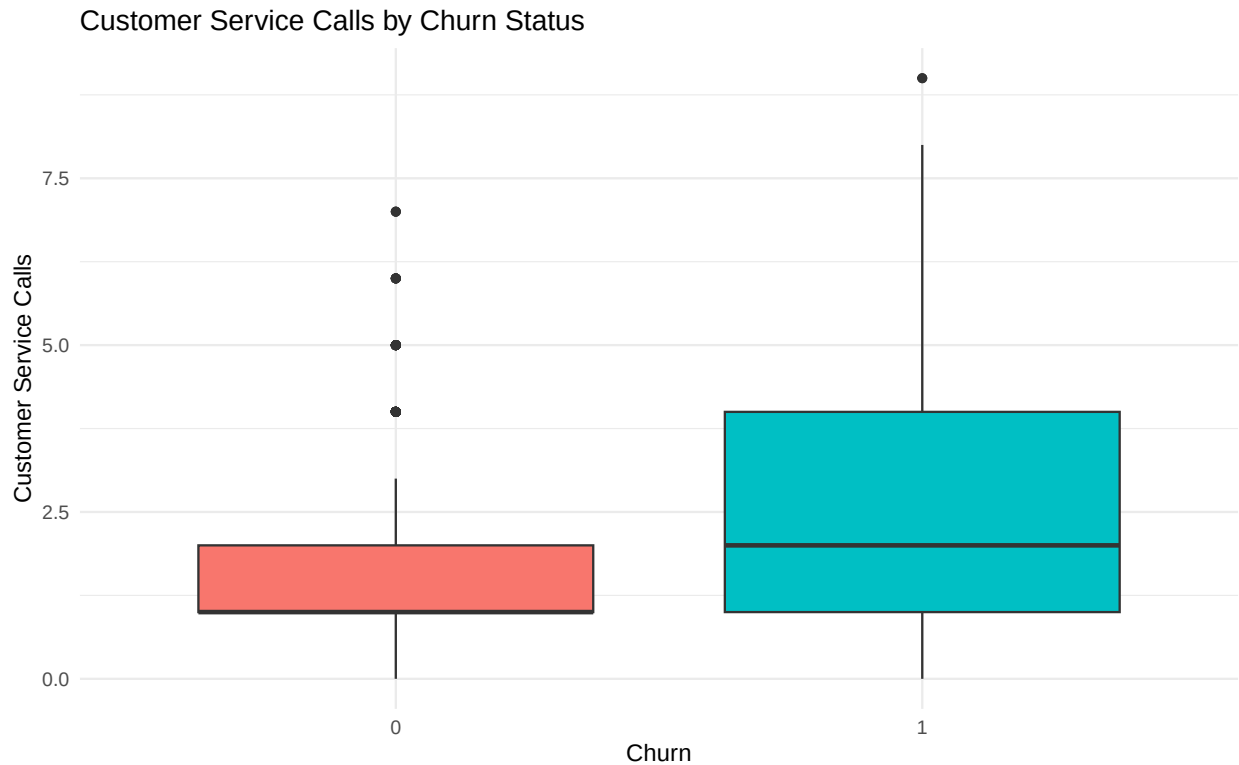
Boxplots by Churn Status

```
ggplot(train_clean,
       aes(x = factor(churn),
           y = total_day_minutes,
           fill = factor(churn))) +
  geom_boxplot() +
  labs(
    title = "Total Day Minutes by Churn Status",
    x = "Churn",
    y = "Total Day Minutes"
  ) +
  theme_minimal() +
  theme(
    legend.position = "none"
  )
```



Notes: Customers who churned generally recorded higher total day minutes than customers who remained with the company. This results suggests that heavier daytime usage may be associated with an increased likelihood of customer churn.

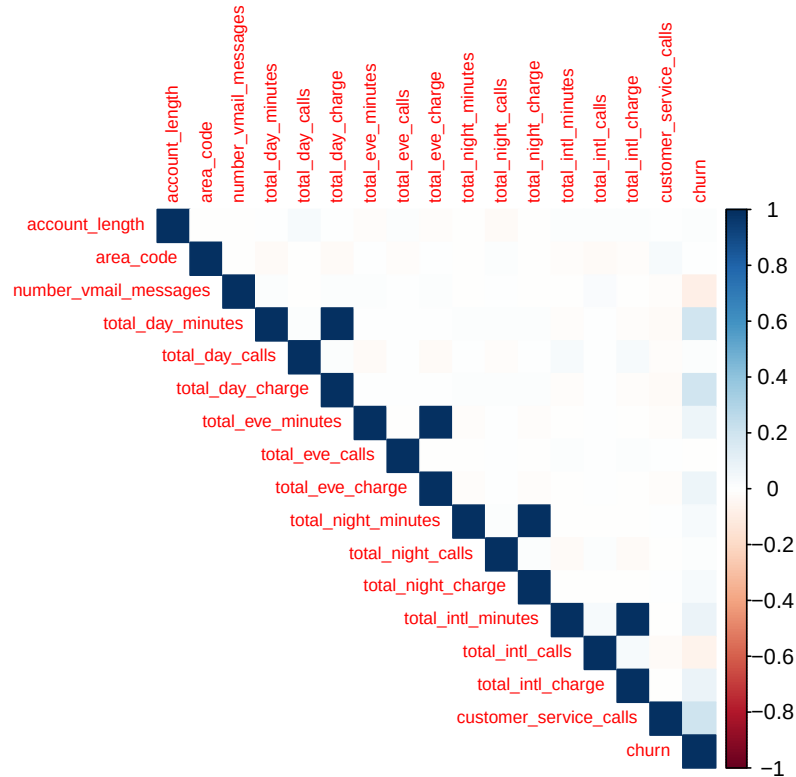
```
ggplot(train_clean,
  aes(x = factor(churn),
    y = customer_service_calls,
    fill = factor(churn))) +
  geom_boxplot() +
  labs(
    title = "Customer Service Calls by Churn Status",
    x = "Churn",
    y = "Customer Service Calls"
  ) +
  theme_minimal() +
  theme(
    legend.position = "none"
  )
```



Notes: Customers who churned tended to make more customer service calls than non-churning customers, with a higher median and greater variability. This result suggests that frequent interactions with customer support may possibly be an indicator of customer dissatisfaction.

Correlation Analysis

```
numeric_data <- train_clean %>%  
  select(where(is.numeric))  
  
cor_matrix <- cor(numeric_data)  
  
corrplot(  
  cor_matrix,  
  method = "color",  
  type = "upper",  
  tl.cex = 0.7  
)
```



Notes: The correlation heatmap shows strong positive correlations between usage variables and their corresponding charges, which is expected since charges are calculated directly from usage. Most other predictor variables exhibit relatively weak correlations, suggesting limited multicollinearity among the remaining features.

Class Imbalance Examination

```
prop.table(table(train_clean$churn)) * 100
```

```
##
##      0      1
## 85.44636 14.55364
```

Notes: The class distribution confirms that 85.45% of customers did not churn, while 14.55% did. Although the imbalance is moderate, evaluation metrics such as Precision, Recall, F1-score, and ROC-AUC will provide a more reliable assessment of model performance than accuracy alone.

3.0 Modeling and Comparison

Regression-Based Models

Model 1: Logistic Regression

Model Construction

```
log_model <- glm(
  churn ~ .,
  data = train_processed,
  family = binomial
)

summary(log_model)
```

```
##
## Call:
## glm(formula = churn ~ ., family = binomial, data = train_processed)
##
## Coefficients: (2 not defined because of singularities)
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -7.724e+00  1.254e+00  -6.161 7.25e-10 ***
## account_length    8.890e-04  1.568e-03   0.567  0.57062
## area_code       -5.401e-04  1.472e-03  -0.367  0.71369
## international_planNo -2.104e+00  1.599e-01 -13.162 < 2e-16 ***
## international_planYes      NA          NA      NA      NA
## voice_mail_planNo    2.065e+00  6.569e-01   3.143  0.00167 **
## voice_mail_planYes      NA          NA      NA      NA
## number_vmail_messages  3.848e-02  2.066e-02   1.863  0.06253 .
## total_day_minutes   -5.274e-01  3.672e+00  -0.144  0.88580
## total_day_calls      2.872e-03  3.104e-03   0.925  0.35480
## total_day_charge      3.176e+00  2.160e+01   0.147  0.88310
## total_eve_minutes     1.536e+00  1.844e+00   0.833  0.40483
## total_eve_calls      -7.851e-04  3.086e-03  -0.254  0.79920
## total_eve_charge     -1.800e+01  2.169e+01  -0.830  0.40656
## total_night_minutes    4.176e-03  9.829e-01   0.004  0.99661
## total_night_calls      2.001e-03  3.170e-03   0.631  0.52795
## total_night_charge     -3.076e-02  2.184e+01  -0.001  0.99888
## total_intl_minutes     -3.566e+00  5.937e+00  -0.601  0.54810
## total_intl_calls      -1.206e-01  2.886e-02  -4.180 2.92e-05 ***
## total_intl_charge      1.358e+01  2.199e+01   0.617  0.53691
## customer_service_calls  5.075e-01  4.412e-02  11.503 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 2212.2  on 2665  degrees of freedom
## Residual deviance: 1729.4  on 2647  degrees of freedom
## AIC: 1767.4
```

```
##  
## Number of Fisher Scoring iterations: 6
```

Notes: Logistic Regression was selected because it is a widely used baseline classification model for binary outcomes such as customer churn. The model estimates the probability that a customer will churn based on the available predictor variables.

Predict on the Testing Dataset

```
log_prob <- predict(  
  log_model,  
  newdata = test_processed,  
  type = "response"  
)  
  
log_pred <- ifelse(log_prob >= 0.5, 1, 0)
```

Notes: The trained logistic regression model is applied to the testing dataset to estimate the probability of customer churn. Customers with predicted probabilities greater than or equal to 0.50 are classified as churners.

Confusion Matrix

```
confusionMatrix(  
  factor(log_pred),  
  factor(test_processed$churn),  
  positive = "1"  
)  
  
## Confusion Matrix and Statistics  
##  
##           Reference  
## Prediction    0    1  
##           0 552  79  
##           1  20  16  
##  
##           Accuracy : 0.8516  
##           95% CI : (0.8223, 0.8777)  
##    No Information Rate : 0.8576  
##    P-Value [Acc > NIR] : 0.6944  
##  
##           Kappa : 0.1801  
##  
##    McNemar's Test P-Value : 5.569e-09  
##  
##           Sensitivity : 0.16842  
##           Specificity : 0.96503  
##    Pos Pred Value : 0.44444  
##    Neg Pred Value : 0.87480
```

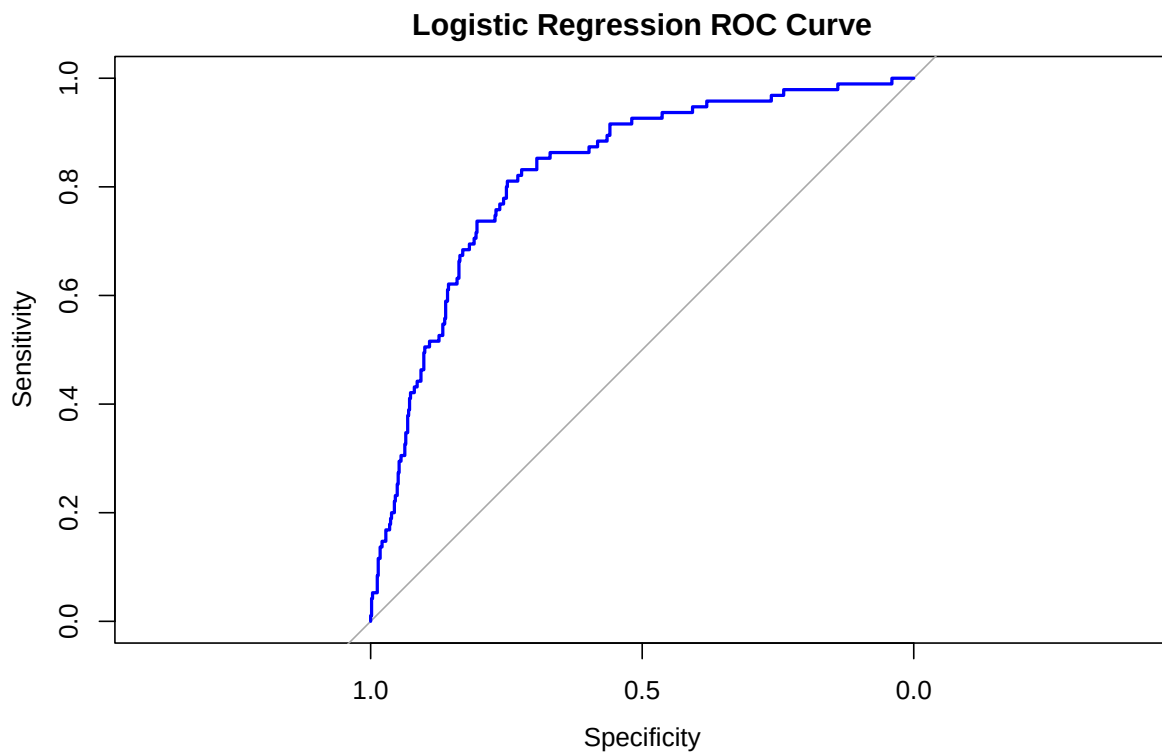
```
##           Prevalence : 0.14243
##           Detection Rate : 0.02399
##           Detection Prevalence : 0.05397
##           Balanced Accuracy : 0.56673
##
##           'Positive' Class : 1
##
```

Notes: The confusion matrix was selected as it needed to summarize the classification performance by comparing the predicted and actual churn outcomes. Additionally it also provides the basis for calculating several evaluation metrics, including Accuracy, Precision, Recall, and F1-score.

ROC Curve and AUC

```
log_roc <- roc(
  test_processed$churn,
  log_prob
)

plot(
  log_roc,
  main = "Logistic Regression ROC Curve",
  col = "blue"
)
```



```
auc(log_roc)
```

```
## Area under the curve: 0.8255
```

Notes: The ROC curve illustrates the trade-off between the true positive rate and false positive rate across different classification thresholds. Furthermore, the Area Under the Curve (AUC) also measures the model's overall ability to distinguish between customers who churn and those who do not. Due to this both of models were selected for this study.

Variable Importance (Coefficients)

```
coef(summary(log_model)) %>%  
  as.data.frame() %>%  
  arrange(desc(abs(Estimate))) %>%  
  head(10) %>%  
  kable(  
    caption = "Top Logistic Regression Coefficients"  
  )
```

Table 4: Top Logistic Regression Coefficients

	Estimate	Std. Error	z value	Pr(> z)
total_eve_charge	-18.0037940	21.6924835	-0.8299554	0.4065640
total_intl_charge	13.5780355	21.9888365	0.6174968	0.5369071
(Intercept)	-7.7240246	1.2537844	-6.1605684	0.0000000
total_intl_minutes	-3.5659356	5.9371976	-0.6006092	0.5481003
total_day_charge	3.1762265	21.6006908	0.1470428	0.8830982
international_planNo	-2.1041396	0.1598649	-13.1619897	0.0000000
voice_mail_planNo	2.0647244	0.6569428	3.1429287	0.0016727
total_eve_minutes	1.5360119	1.8438764	0.8330341	0.4048255
total_day_minutes	-0.5273931	3.6721105	-0.1436213	0.8857996
customer_service_calls	0.5075102	0.0441206	11.5028047	0.0000000

Notes: The regression coefficients indicate the influence of each predictor on the likelihood of customer churn. Variables with larger absolute coefficient values generally have a stronger effect on the model's predictions.

Model 2: Ridge Regression

Model Construction

```
x_train <- model.matrix(  
  churn ~ .,  
  train_processed  
)[, -1]
```

```

y_train <- train_processed$churn

x_test <- model.matrix(
  churn ~ .,
  test_processed
)[-1]

y_test <- test_processed$churn

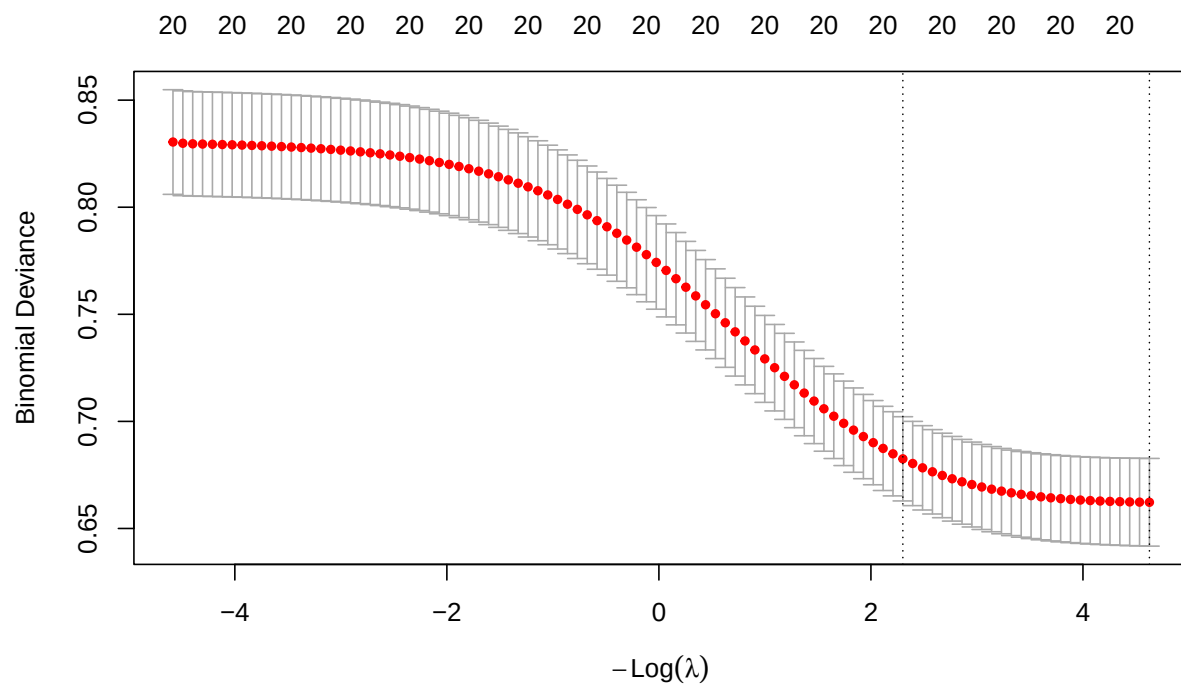
```

```

ridge_model <- cv.glmnet(
  x_train,
  y_train,
  alpha = 0,
  family = "binomial"
)

plot(ridge_model)

```



Notes: Ridge Regression was selected because it directly addresses the multicollinearity through L2 regularization, which makes it suitable for datasets with multiple correlated predictors.

Predictions

```

ridge_prob <- predict(
  ridge_model,
  s = "lambda.min",

```

```

newx = x_test,
type = "response"
)

ridge_pred <- ifelse(
  ridge_prob > 0.5,
  1,
  0
)

```

Notes: The trained Ridge Regression model generates the probability that each customer will churn using the unseen testing dataset. These probabilities are then converted into binary class predictions using a threshold of 0.50, allowing the model's classification performance to be evaluated.

Confusion Matrix

```

confusionMatrix(
  factor(ridge_pred),
  factor(y_test),
  positive = "1"
)

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0 556  80
##           1  16  15
##
##           Accuracy : 0.8561
##           95% CI : (0.8271, 0.8818)
##       No Information Rate : 0.8576
##       P-Value [Acc > NIR] : 0.571
##
##           Kappa : 0.1807
##
##  McNemar's Test P-Value : 1.277e-10
##
##           Sensitivity : 0.15789
##           Specificity : 0.97203
##       Pos Pred Value : 0.48387
##       Neg Pred Value : 0.87421
##           Prevalence : 0.14243
##       Detection Rate : 0.02249
##   Detection Prevalence : 0.04648
##       Balanced Accuracy : 0.56496
##
##       'Positive' Class : 1
##

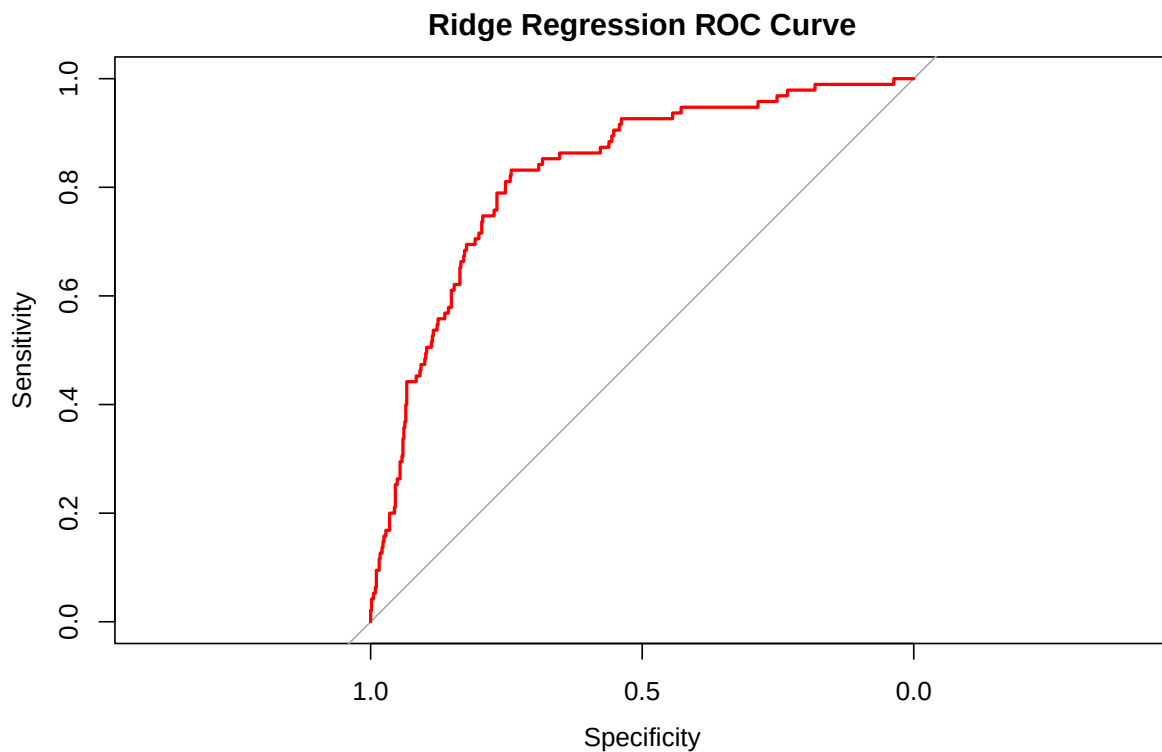
```

Notes: The confusion matrix summarizes the Ridge Regression model's ability to correctly classify customers

as churners or non-churners. Performance metrics such as Accuracy, Precision, Recall, and F1-score provide a comprehensive assessment of the model's predictive capability, particularly in the presence of class imbalance.

ROC Curve

```
ridge_roc <- roc(  
  y_test,  
  as.vector(ridge_prob)  
)  
  
plot(  
  ridge_roc,  
  main = "Ridge Regression ROC Curve",  
  col = "red"  
)
```



```
auc(ridge_roc)
```

```
## Area under the curve: 0.8256
```

Notes: The ROC curve illustrates the trade-off between the true positive rate and false positive rate across different classification thresholds.

Most Important Coefficients

```
coef(
  ridge_model,
  s = "lambda.min"
) %>%
  as.matrix() %>%
  as.data.frame() %>%
  head(15) %>%
  kable(
    caption = "Ridge Regression Coefficients"
  )
```

Table 5: Ridge Regression Coefficients

	lambda.min
(Intercept)	-6.6592797
account_length	0.0008088
area_code	-0.0005618
international_planNo	-0.9997897
international_planYes	0.9919390
voice_mail_planNo	0.4576256
voice_mail_planYes	-0.4475708
number_vmail_messages	0.0028392
total_day_minutes	0.0058430
total_day_calls	0.0024993
total_day_charge	0.0341982
total_eve_minutes	0.0025775
total_eve_calls	-0.0007563
total_eve_charge	0.0300876
total_night_minutes	0.0012556

Notes: The Ridge Regression coefficient results suggests that the contribution of each predictor after the regularization penalty has been applied. Unlike standard logistic regression, Ridge Regression retains all predictor variables while shrinking less important coefficients toward zero, resulting in a more stable and robust model without completely eliminating variables.

Model 3: Tree-based Models

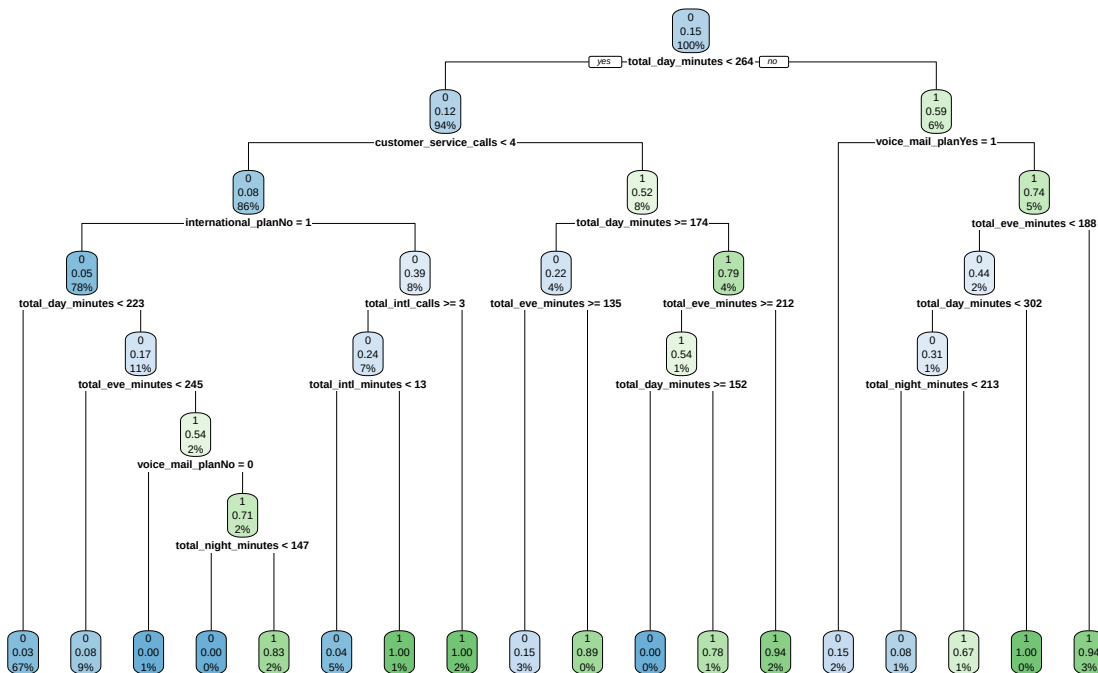
Decision Tree

Model Construction

```
tree_model <- rpart(
  churn ~ .,
  data = train_processed,
  method = "class"
)
```



```
rpart.plot(tree_model)
```



Notes: The decision tree was selected because it is effective at producing simple and interpretable classification rules that clearly show how customer characteristics influence churn predictions. The model was trained using the provided training dataset and evaluated on the separate testing dataset, allowing its performance to be assessed on unseen data. The resulting tree highlights variables such as `total_day_minutes`, `customer_service_calls`, and `international_plan` as the primary decision points for classifying customer churn.

Predictions

```
tree_pred <- predict(
  tree_model,
  newdata = test_processed,
  type = "class"
)
```

Notes: Predictions on the testing dataset were generated to evaluate how well the Decision Tree generalizes beyond the training data. Using an independent test set provides a fair assessment of the model's predictive performance and helps determine its ability to classify new customer records accurately.

Confusion Matrix

```

confusionMatrix(
  factor(tree_pred),
  factor(test_processed$churn),
  positive = "1"
)

## Confusion Matrix and Statistics
##
##              Reference
## Prediction    0    1
##           0 563  20
##           1   9  75
##
##              Accuracy : 0.9565
##              95% CI : (0.9382, 0.9707)
##      No Information Rate : 0.8576
##      P-Value [Acc > NIR] : < 2e-16
##
##              Kappa : 0.813
##
##  McNemar's Test P-Value : 0.06332
##
##              Sensitivity : 0.7895
##              Specificity : 0.9843
##              Pos Pred Value : 0.8929
##              Neg Pred Value : 0.9657
##              Prevalence : 0.1424
##              Detection Rate : 0.1124
##      Detection Prevalence : 0.1259
##              Balanced Accuracy : 0.8869
##
##              'Positive' Class : 1
##

```

Notes: The confusion matrix was selected because it summarizes the model's classification performance through Accuracy, Precision, Recall, and F1-score. The Decision Tree achieved an Accuracy of 95.65%, Precision of 89.29%, and Recall of 78.95%, indicating strong overall performance while successfully identifying most customers who churned. These results demonstrate that the model performs substantially better than the baseline classification rate.

ROC Curve

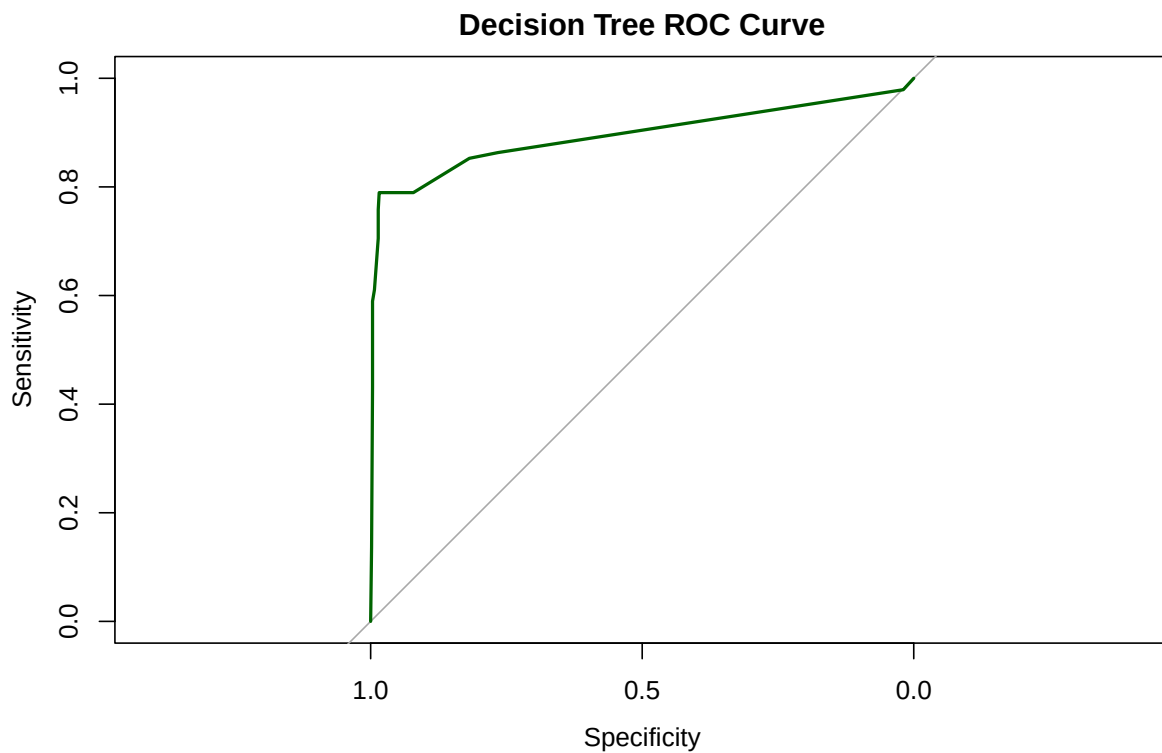
```

tree_prob <- predict(
  tree_model,
  newdata = test_processed,
  type = "prob"
)[,2]

tree_roc <- roc(
  test_processed$churn,
  tree_prob

```

```
)
plot(
  tree_roc,
  main = "Decision Tree ROC Curve",
  col = "darkgreen"
)
```



```
auc(tree_roc)
```

```
## Area under the curve: 0.8946
```

Notes: The ROC curve was selected because it evaluates the model across all possible classification thresholds instead of relying on a single cutoff value. The Decision Tree achieved an ROC-AUC of 0.8946, this shows that it has an effective ability to distinguish between customers who churn and those who remain with the company.

Variable Importance

```
tree_model$variable.importance %>%
  sort(decreasing = TRUE) %>%
  head(10) %>%
  kable(
    caption = "Decision Tree Variable Importance"
  )
```

Table 6: Decision Tree Variable Importance

	x
total_day_charge	130.72886
total_day_minutes	130.72886
customer_service_calls	71.77670
total_eve_charge	58.49833
total_eve_minutes	58.49833
total_intl_charge	55.10423
total_intl_minutes	55.10423
international_planNo	47.95244
international_planYes	47.95244
total_intl_calls	41.51636

Notes: The variable importance results identify total_day_minutes, total_day_charge, and customer_service_calls as the strongest predictors of customer churn. This suggests that customer usage patterns and interactions with customer support have the greatest influence on the Decision Tree's classification decisions.

Model 4: Random Forest

Model Construction

```
rf_model <- randomForest(
  factor(churn) ~ .,
  data = train_processed,
  ntree = 500,
  importance = TRUE
)

rf_model
```

```
##
## Call:
## randomForest(formula = factor(churn) ~ ., data = train_processed, ntree = 500, importance = TRUE)
##           Type of random forest: classification
##           Number of trees: 500
## No. of variables tried at each split: 4
##
##           OOB estimate of  error rate: 4.69%
## Confusion matrix:
##           0    1 class.error
## 0 2259  19  0.00834065
## 1  106 282  0.27319588
```

Notes: Random Forest was selected because it combines multiple Decision Trees to improve prediction accuracy while reducing overfitting. The model was trained using 500 decision trees, and its low out-of-bag error rate of 4.69% indicates good generalization performance before testing on unseen data.

Predictions

```
rf_pred <- predict(  
  rf_model,  
  newdata = test_processed,  
  type = "class"  
)
```

Notes: Predictions were generated using the testing dataset to evaluate the Random Forest model on new customer records. This ensures that the reported performance reflects the model's ability to generalize rather than simply memorizing the training data.

Confusion Matrix

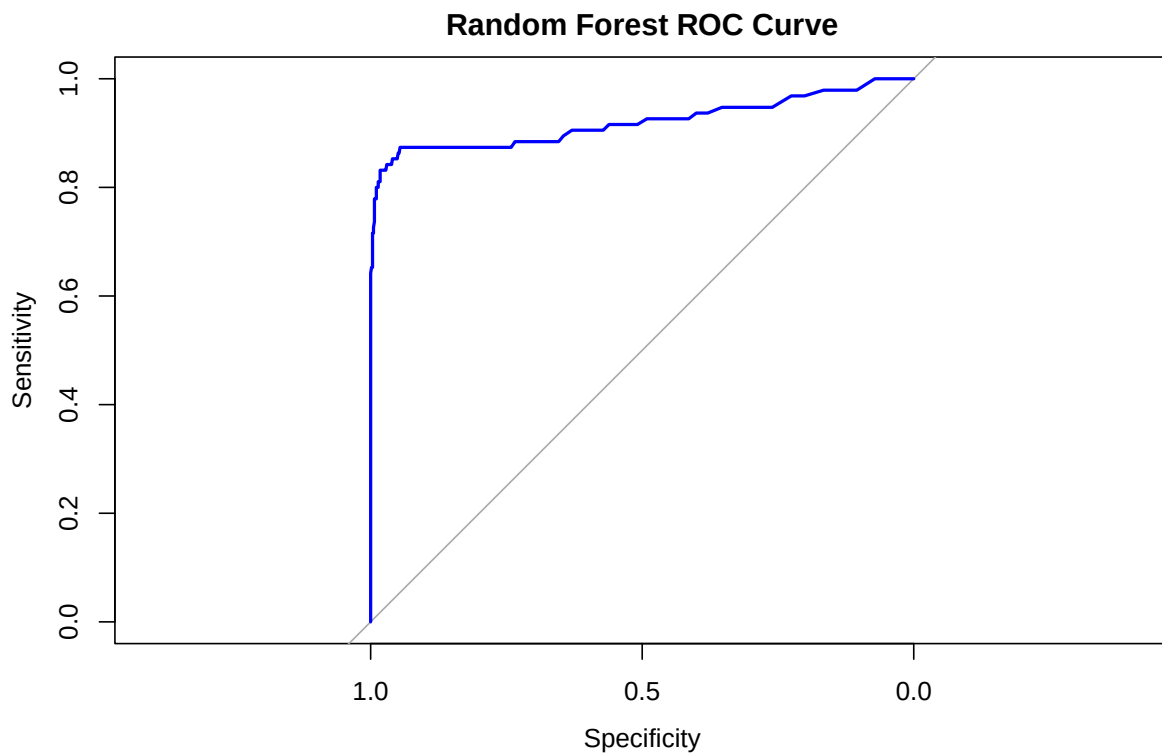
```
confusionMatrix(  
  rf_pred,  
  factor(test_processed$churn),  
  positive = "1"  
)
```

```
## Confusion Matrix and Statistics  
##  
##           Reference  
## Prediction    0    1  
##           0 568   25  
##           1   4   70  
##  
##           Accuracy : 0.9565  
##           95% CI : (0.9382, 0.9707)  
##           No Information Rate : 0.8576  
##           P-Value [Acc > NIR] : < 2.2e-16  
##  
##           Kappa : 0.8039  
##  
##           McNemar's Test P-Value : 0.0002041  
##  
##           Sensitivity : 0.7368  
##           Specificity : 0.9930  
##           Pos Pred Value : 0.9459  
##           Neg Pred Value : 0.9578  
##           Prevalence : 0.1424  
##           Detection Rate : 0.1049  
##           Detection Prevalence : 0.1109  
##           Balanced Accuracy : 0.8649  
##  
##           'Positive' Class : 1  
##
```

Notes: The confusion matrix was selected because it provides a comprehensive evaluation of the Random Forest classifier through Accuracy, Precision, Recall, and F1-score. The model achieved an Accuracy of 95.65%, Precision of 94.59%, and Recall of 73.68%, demonstrating excellent overall predictive performance while maintaining a very low false positive rate.

ROC Curve

```
rf_prob <- predict(  
  rf_model,  
  newdata = test_processed,  
  type = "prob"  
)[,2]  
  
rf_roc <- roc(  
  test_processed$churn,  
  rf_prob  
)  
  
plot(  
  rf_roc,  
  main = "Random Forest ROC Curve",  
  col = "blue"  
)
```



```
auc(rf_roc)
```

```
## Area under the curve: 0.9201
```

Notes: The ROC curve evaluates the Random Forest model across different classification thresholds to measure its discriminative ability. The model achieved an ROC-AUC of 0.9201, this shows that is quite effective in distinguishing customers who churn from those who do not.

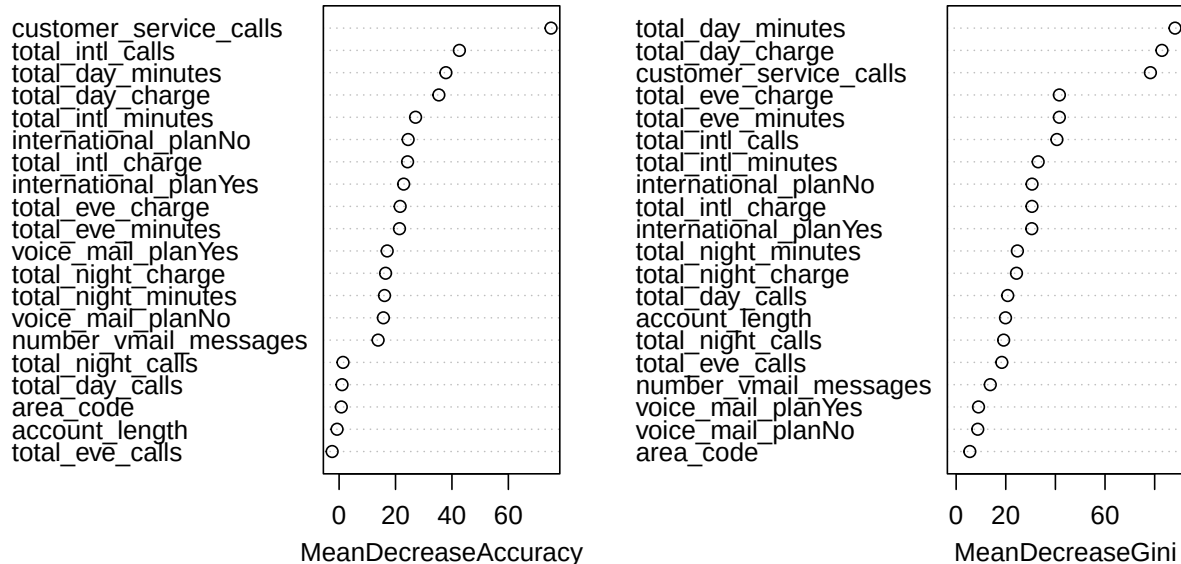
Variable Importance

```
importance(rf_model)
```

##		0	1	MeanDecreaseAccuracy
##	account_length	0.5442316	-2.5374765	-0.7195956
##	area_code	0.6504543	0.5226718	0.7805755
##	international_planNo	22.6038406	24.5127068	24.4760782
##	international_planYes	20.8237665	23.2727412	22.8834954
##	voice_mail_planNo	12.6644870	15.1578102	15.6988662
##	voice_mail_planYes	14.4027241	16.5514738	17.0493141
##	number_vmail_messages	9.8056884	15.2076278	13.8287646
##	total_day_minutes	28.2616203	31.3277045	37.8636576
##	total_day_calls	0.7508048	0.8501027	1.0414497
##	total_day_charge	27.0003307	30.4328297	35.3638102
##	total_eve_minutes	18.0703302	20.8698569	21.4067702
##	total_eve_calls	-1.8410253	-1.7381947	-2.4665329
##	total_eve_charge	17.9965283	19.5997454	21.6395583
##	total_night_minutes	15.6415436	-0.1776933	16.1066844
##	total_night_calls	2.4575781	-1.8976412	1.3870242
##	total_night_charge	15.9333176	0.1447642	16.4745341
##	total_intl_minutes	21.2808362	20.0650601	27.1107935
##	total_intl_calls	29.9311373	42.9886712	42.6460370
##	total_intl_charge	20.2804047	17.0848069	24.2699346
##	customer_service_calls	55.4549565	75.2547992	75.1332302
##		MeanDecreaseGini		
##	account_length	19.897880		
##	area_code	5.516599		
##	international_planNo	30.593771		
##	international_planYes	30.480252		
##	voice_mail_planNo	8.707947		
##	voice_mail_planYes	9.014983		
##	number_vmail_messages	13.760018		
##	total_day_minutes	88.219768		
##	total_day_calls	20.785040		
##	total_day_charge	82.938135		
##	total_eve_minutes	41.596270		
##	total_eve_calls	18.413151		
##	total_eve_charge	41.598294		
##	total_night_minutes	24.685666		
##	total_night_calls	19.145285		
##	total_night_charge	24.319615		
##	total_intl_minutes	33.053894		
##	total_intl_calls	40.614660		
##	total_intl_charge	30.556235		
##	customer_service_calls	78.294666		

```
varImpPlot(rf_model)
```

rf_model



Notes: The feature importance analysis shows that `customer_service_calls`, `total_day_minutes`, `total_day_charge`, and `total_intl_calls` are the most influential predictors of customer churn. These findings indicate that customer service interactions and communication usage play a significant role in the Random Forest's classification decisions.

Model 5: Gradient Boosting Classifier

Model Construction

```
gbm_model <- gbm(
  churn ~ .,
  data = train_processed,
  distribution = "bernoulli",
  n.trees = 200,
  interaction.depth = 3,
  shrinkage = 0.05,
  cv.folds = 5,
  verbose = FALSE
)
```

Notes: Gradient Boosting was selected because it sequentially builds decision trees that correct the errors made by previous trees, often producing highly accurate predictive models.

Predictions

```
gbm_prob <- predict(
  gbm_model,
  newdata = test_processed,
  n.trees = 200,
  type = "response"
)

gbm_pred <- ifelse(
  gbm_prob > 0.5,
  1,
  0
)
```

Notes: Predictions were generated on the testing dataset to evaluate the Gradient Boosting model's performance on unseen customer records.

Confusion Matrix

```
confusionMatrix(
  factor(gbm_pred),
  factor(test_processed$churn),
  positive = "1"
)
```

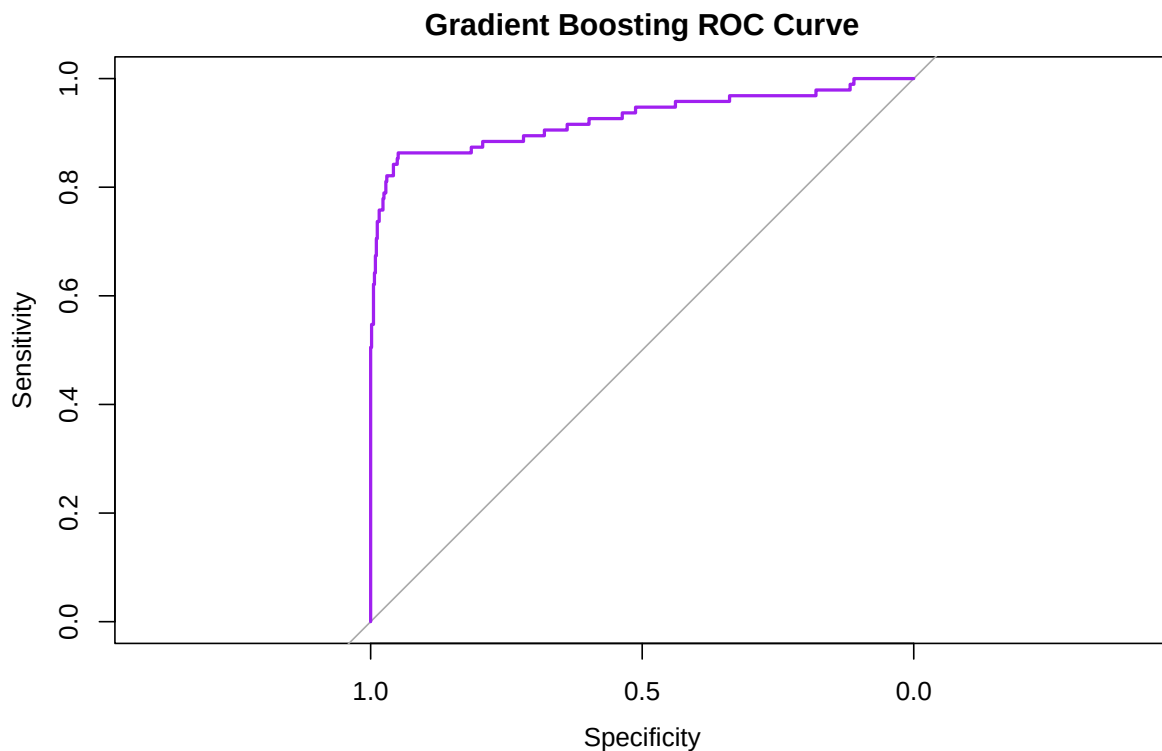
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0 566  28
##           1   6  67
##
##           Accuracy : 0.949
##           95% CI : (0.9295, 0.9644)
##           No Information Rate : 0.8576
##           P-Value [Acc > NIR] : 2.125e-14
##
##           Kappa : 0.769
##
##  Mcnemar's Test P-Value : 0.0003164
##
##           Sensitivity : 0.7053
##           Specificity : 0.9895
##           Pos Pred Value : 0.9178
##           Neg Pred Value : 0.9529
##           Prevalence : 0.1424
##           Detection Rate : 0.1004
##           Detection Prevalence : 0.1094
##           Balanced Accuracy : 0.8474
```

```
##  
##      'Positive' Class : 1  
##
```

Notes: The confusion matrix was selected because it summarizes the overall classification performance using Accuracy, Precision, Recall, and F1-score. The Gradient Boosting model achieved an Accuracy of 94.90%, Precision of 91.78%, and Recall of 70.53%, a high showing performance whilst also properly identifying the majority of churning customers.

ROC Curve

```
gbm_roc <- roc(  
  test_processed$churn,  
  gbm_prob  
)  
  
plot(  
  gbm_roc,  
  main = "Gradient Boosting ROC Curve",  
  col = "purple"  
)
```



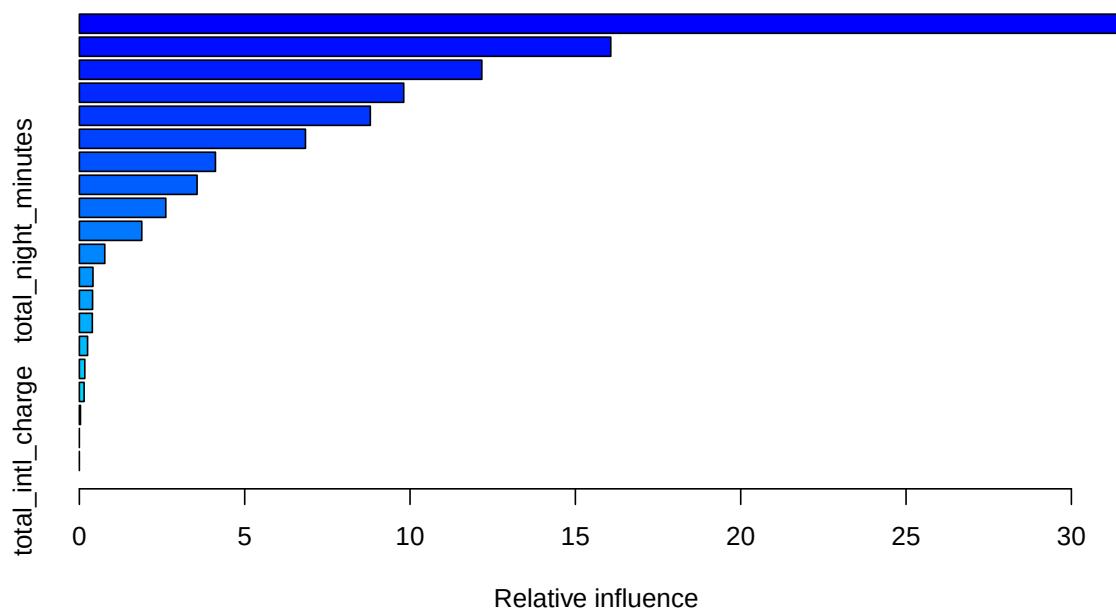
```
auc(gbm_roc)
```

```
## Area under the curve: 0.9258
```

Notes: The ROC curve was selected because it provides a threshold-independent evaluation of the classifier's ability to separate churning and non-churning customers. The ROC-AUC of 0.9258 is the highest among the evaluated models, indicating that Gradient Boosting provides the strongest overall discrimination between the two customer groups.

Feature Importance

```
summary(gbm_model)
```



```
##                                var      rel.inf
## total_day_minutes              total_day_minutes 31.57485941
## customer_service_calls         customer_service_calls 16.06958558
## total_eve_minutes              total_eve_minutes 12.17053374
## total_intl_calls               total_intl_calls  9.80935583
## total_intl_minutes             total_intl_minutes  8.79910397
## international_planYes          international_planYes 6.83804363
## international_planNo           international_planNo 4.11550487
## voice_mail_planYes             voice_mail_planYes 3.55982400
## voice_mail_planNo             voice_mail_planNo 2.61356620
## total_night_minutes            total_night_minutes 1.88744531
## total_night_calls              total_night_calls 0.76872176
## total_day_calls                total_day_calls 0.41129464
## number_vmail_messages          number_vmail_messages 0.39994266
## account_length                 account_length 0.39276833
## total_night_charge             total_night_charge 0.24818129
## total_eve_calls                total_eve_calls 0.16461325
```

## total_eve_charge	total_eve_charge	0.14322574
## area_code	area_code	0.03342978
## total_day_charge	total_day_charge	0.00000000
## total_intl_charge	total_intl_charge	0.00000000

Notes: The feature importance results show that total_day_minutes has the greatest relative influence on customer churn prediction, followed by customer_service_calls, total_eve_minutes, and total_intl_calls. These results indicate that customer usage behavior and service interactions are the most influential factors driving the Gradient Boosting model's predictions.

4.0 Conclusion and Interpretation

In this assessment five machine learning models were directly compared to predict customer churn using the Orange Telecom Churn Dataset. From the evaluation metrics, Gradient Boosting demonstrated the most effective predictive performance achieving the highest ROC-AUC while also maintaining high accuracy and strong classification performance. Its boosting approach enabled the model to capture complex relationships within the data, making it the most effective model for identifying customers who are likely to churn.

The comparison of the models also illustrates the bias-variance trade-off in machine learning. Simpler models such as Logistic Regression and Decision Trees generally have higher bias but are easier to interpret, whereas ensemble methods such as Random Forest and Gradient Boosting reduce variance and improve predictive accuracy by combining multiple decision trees. Due to this, ensemble models typically generalize better to unseen data but require greater computational resources and are less transparent.

Another point that should be noted is the proper balance between interpretability and predictive performance. Logistic Regression provides easily interpretable coefficients that explain how individual variables influence customer churn, making it valuable for business analysis. In contrast to this, Random Forest and Gradient Boosting achieve superior predictive performance through more complex decision-making processes.

Concluding this study, predictive churn models provide significant value to telecommunications companies by supporting data-driven decision making. Machine learning models enable organizations and companies to reduce customer attrition, improve customer satisfaction, and develop more effective business plans.